

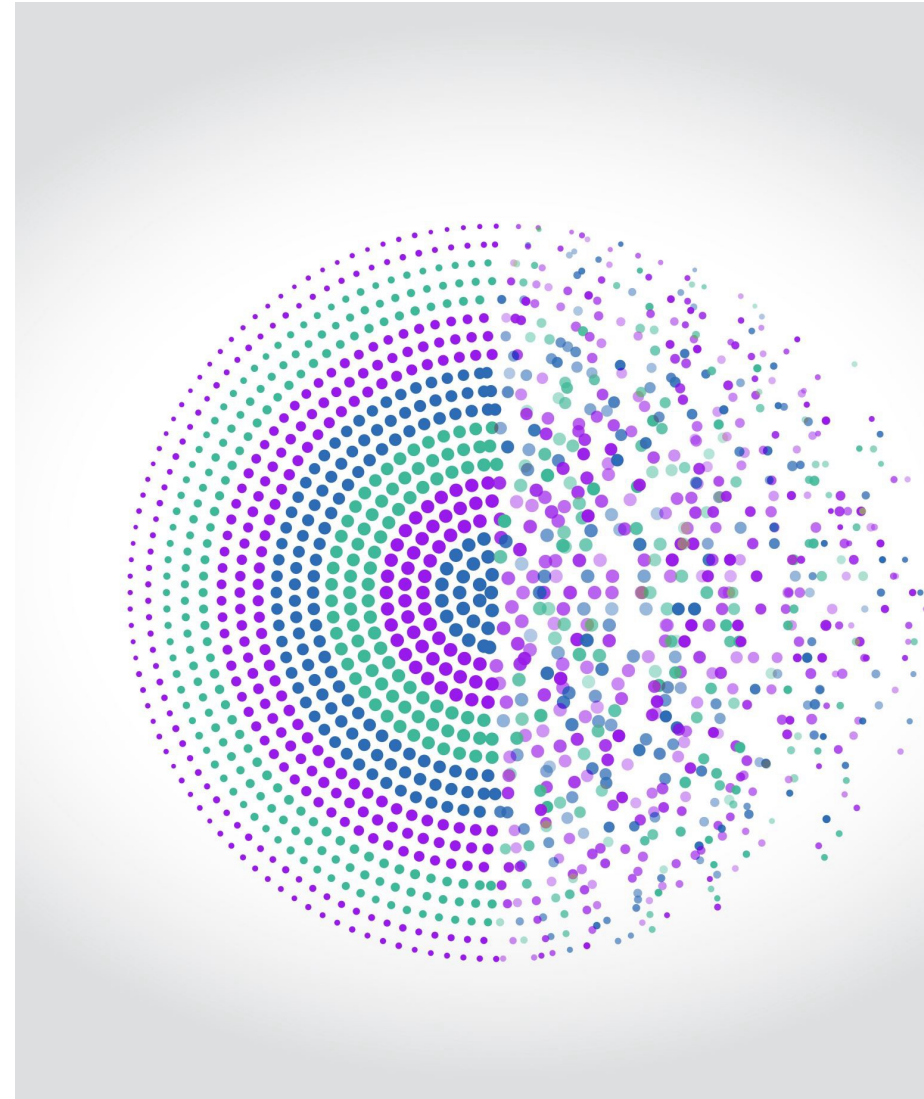
Datenbankmodellierung

Prof. Dr. Michael Kaufmann

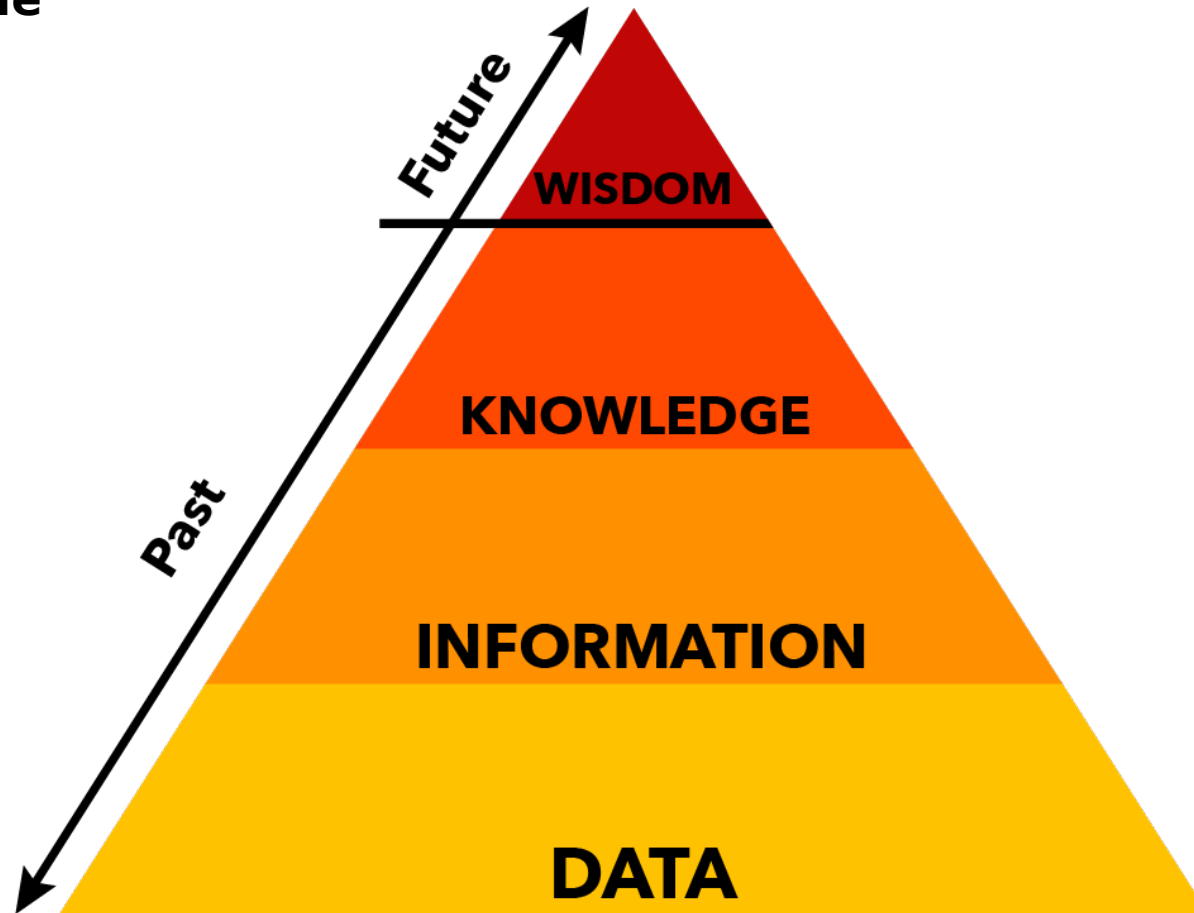
Informatik

11. September 2025

FH Zentralschweiz

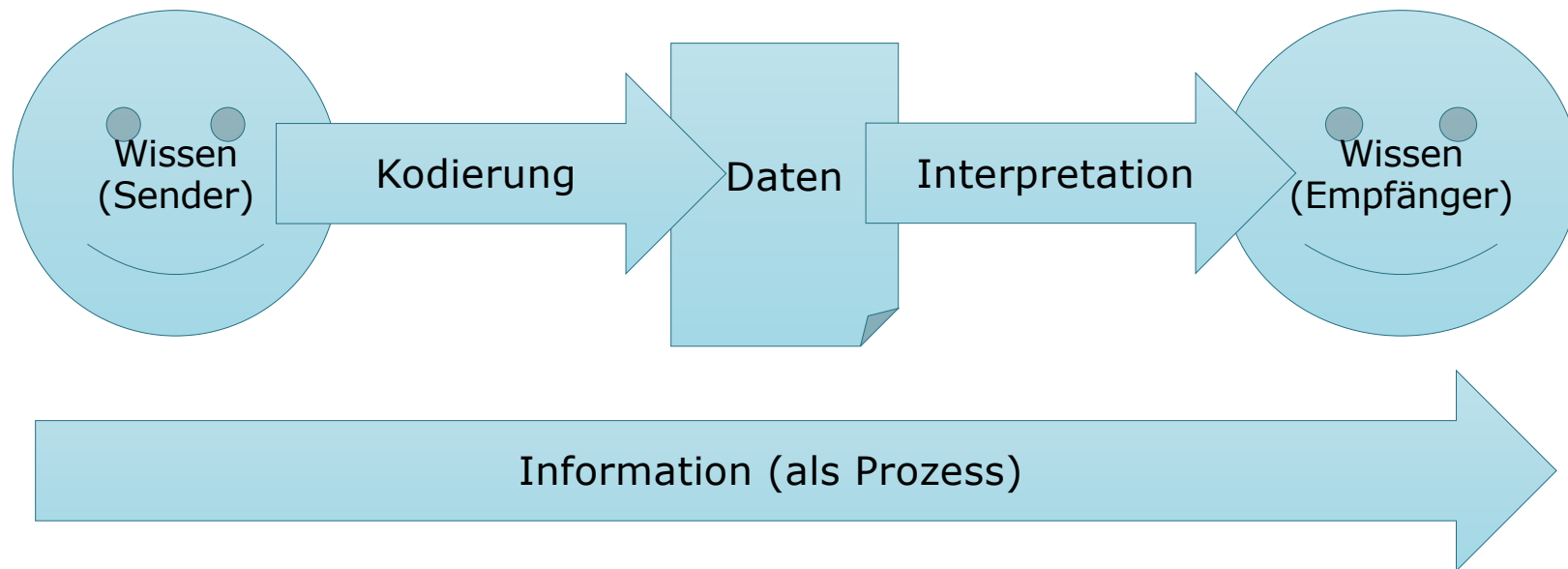


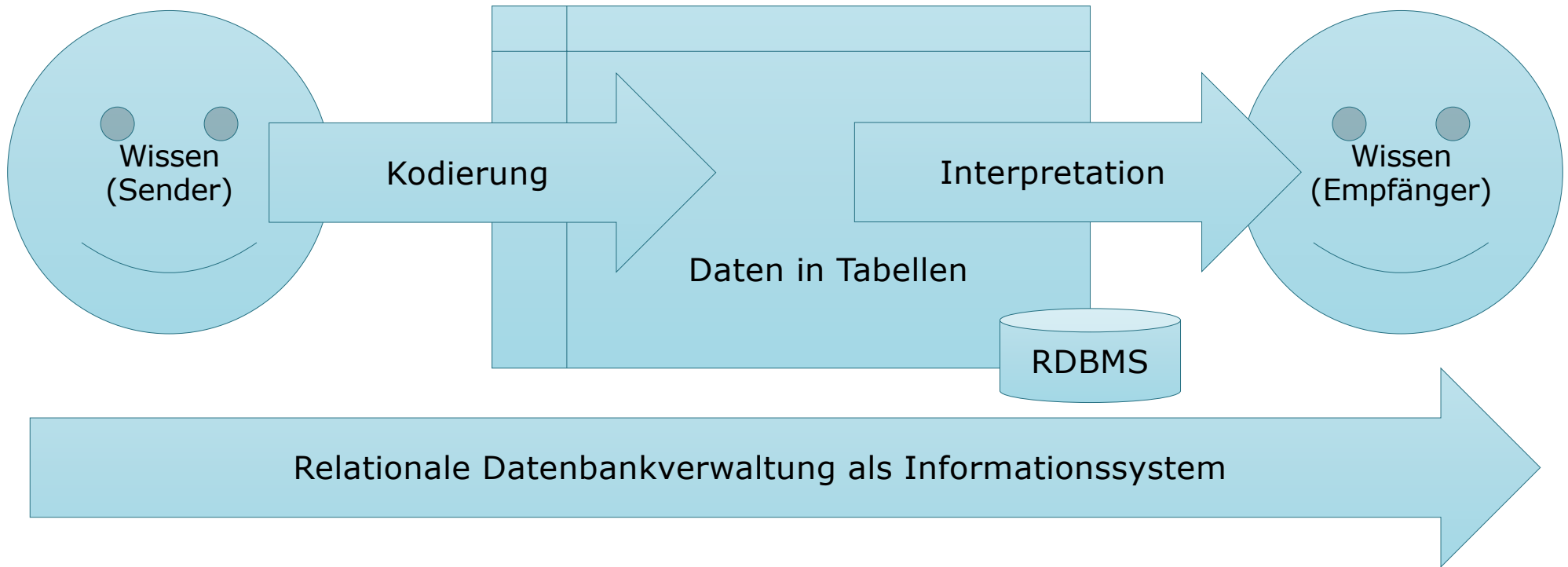
DIKW-Pyramide



"Dieses Foto" von Unbekannter Autor ist lizenziert gemäß [CC BY](#).

Kodierung von Wissen in Daten





Die Abbildung von Wissen in Datenbanktabellen

Was sind die wichtigsten Anforderungen für die effektive Speicherung von Wissen in Tabellen, damit eine Interpretation durch den Empfänger möglich ist?

Was sind die Ziele? Worauf müssen Sie besonders achten?

Tabellendefinition

Eine *Tabelle* oder *Relation* bezieht sich auf eine Reihe von Tupeln (eine Reihe geordneter, unveränderlicher Objekte), die als Tabelle dargestellt werden.

Tabellenname: Eine Tabelle hat einen eindeutigen Namen.

Attributname: Ist eindeutig; bezeichnet eine bestimmte Spalte mit dem gewünschten Attribut.

Keine Spaltenreihenfolge: Die Anzahl der Attribute ist unbegrenzt; die Spaltenreihenfolge hat keine Bedeutung.

Keine Zeilenreihenfolge: Die Zeilennummer in einer Tabelle ist beliebig; die Tupelreihenfolge hat keine Bedeutung.

Identifikationsschlüssel: Eines der Attribute oder eine Kombination davon identifiziert die Tupel in der Tabelle eindeutig und wird als Identifikationsschlüssel bezeichnet.

Diagram illustrating the structure of a table (relation) named **EMPLOYEE**. The table consists of columns (attributes) and rows (records or tuples).

<i>E#</i>	Name	City
E19	Stewart	Stow
E4	Bell	Kent
E1	Murphy	Kent
E7	Howard	Cleveland

Annotations:

- Column (or attribute):** Points to the **Name** column.
- Data value:** Points to the value **Bell** in the **Name** column.
- Record (row or tuple):** Points to the row containing **E4**, **Bell**, and **Kent**.

Datenbankmodellierung

Konzeptionell
Datenmodell:



Datenbankschema: (SQL)

```
CREATE TABLE Employee (
  E# INTEGER,
  Name TEXT,
  City TEXT
  D#_Sub INTEGER )

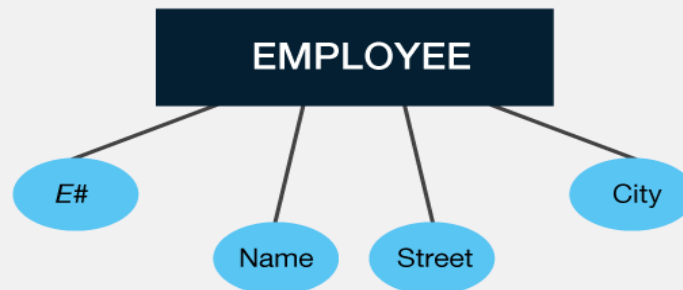
CREATE TABLE Department (
  D# INTEGER,
  Dept_Name TEXT )
```

Was ist in Tabellen gespeichert?

Konzeptionelles Modell Teil 1: Entitäten

Entity:	Employee Murphy, lives on Morris Road in Kent
Entity set:	Set of all employees with the attributes Name, Street, and City
Identification key:	Employee number as an artificial key

Representation in the entity-relationship model

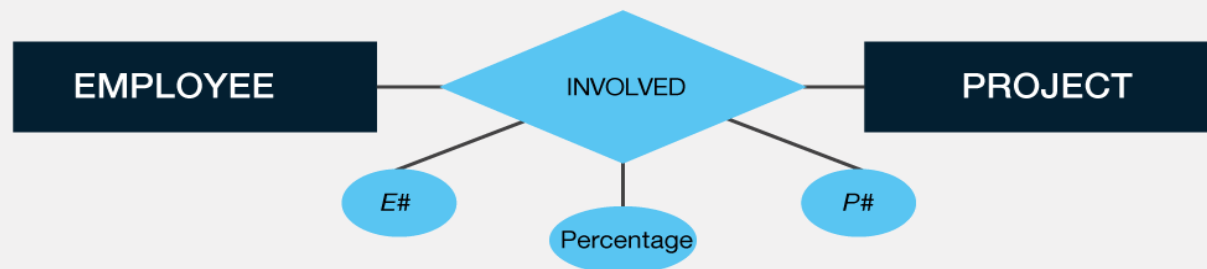


Was ist in Tabellen gespeichert?

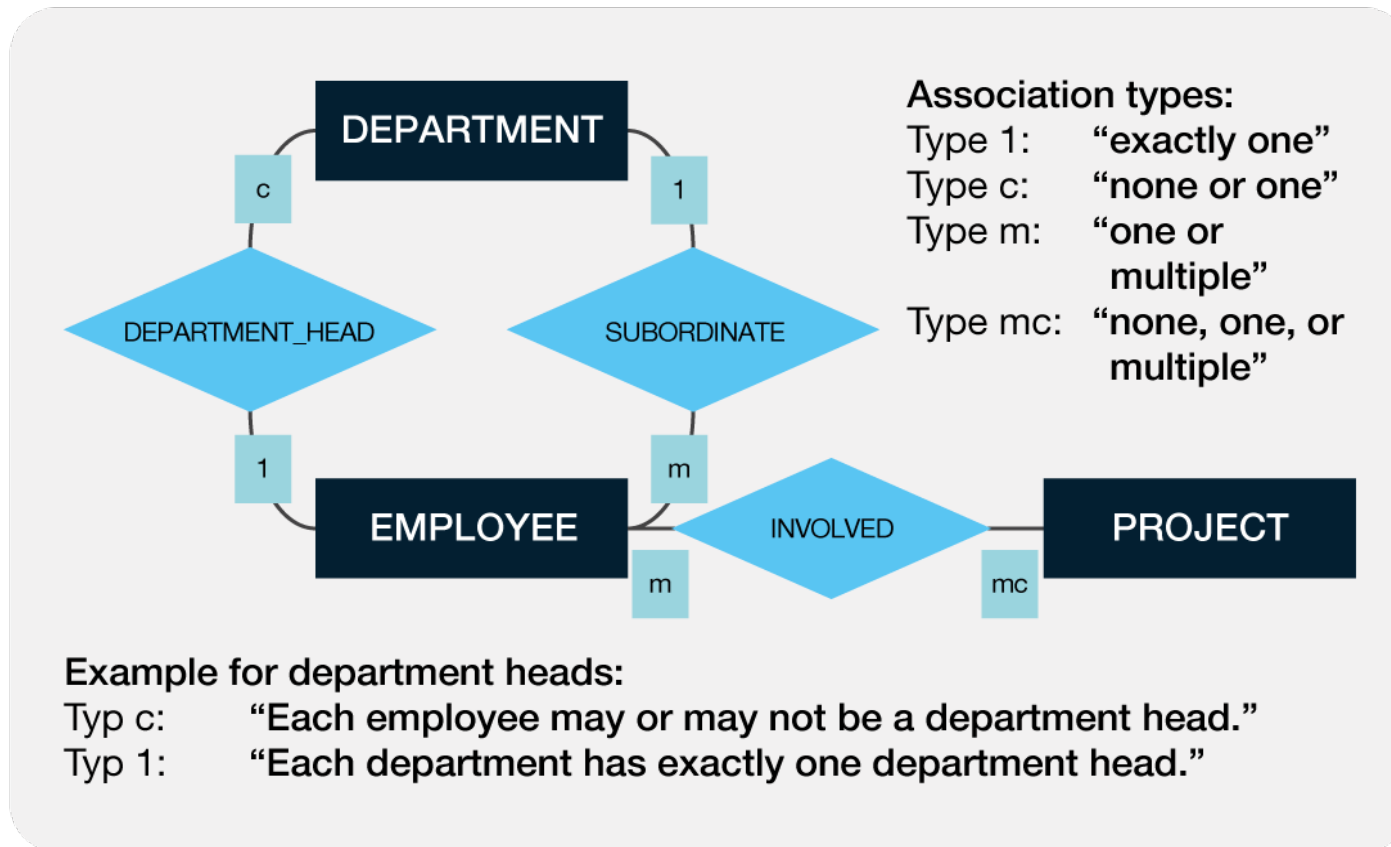
Konzeptionelles Modell Teil 2: Beziehungen

Relationship:	Employee Murphy spends 70 % of their time working on project P17
Relationship set:	Set of all employee project involvements with the attributes Employee number, Project number, and Percentage
Identification key:	Concatenated key consisting of employee number and project number

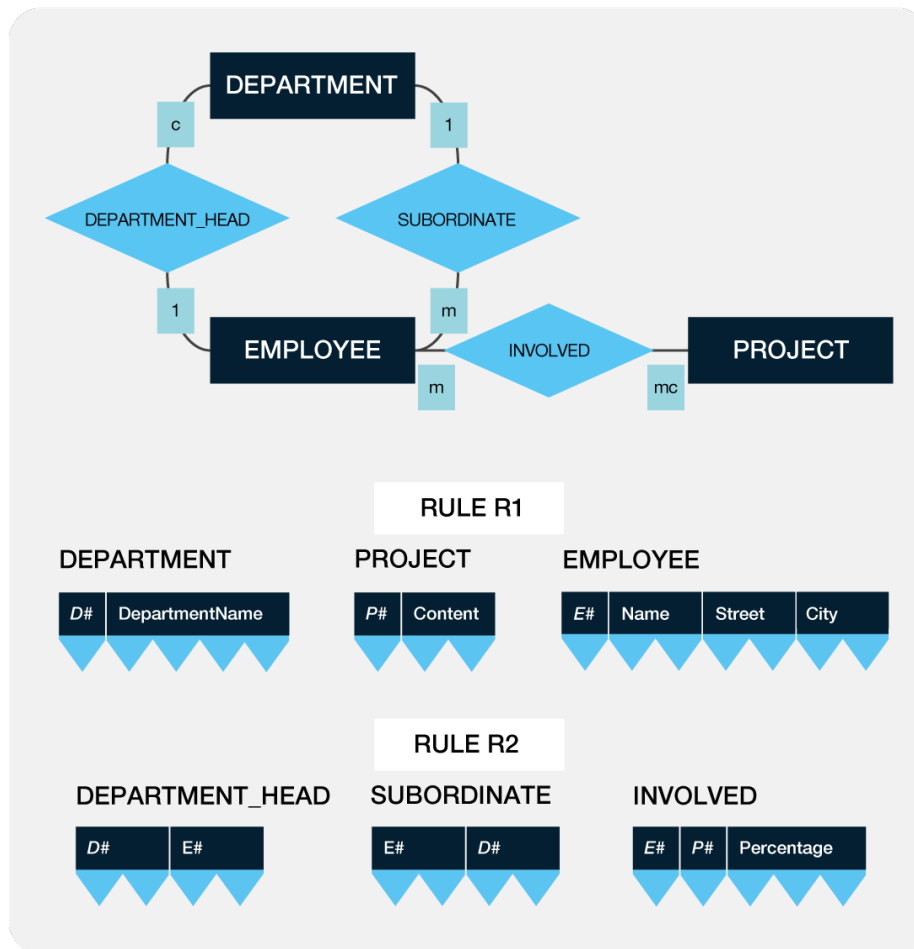
Representation in the entity-relationship model



Ein Entity-Relationship-Modell



Zuordnung von Entitäten und Beziehungssätzen in Tabellen

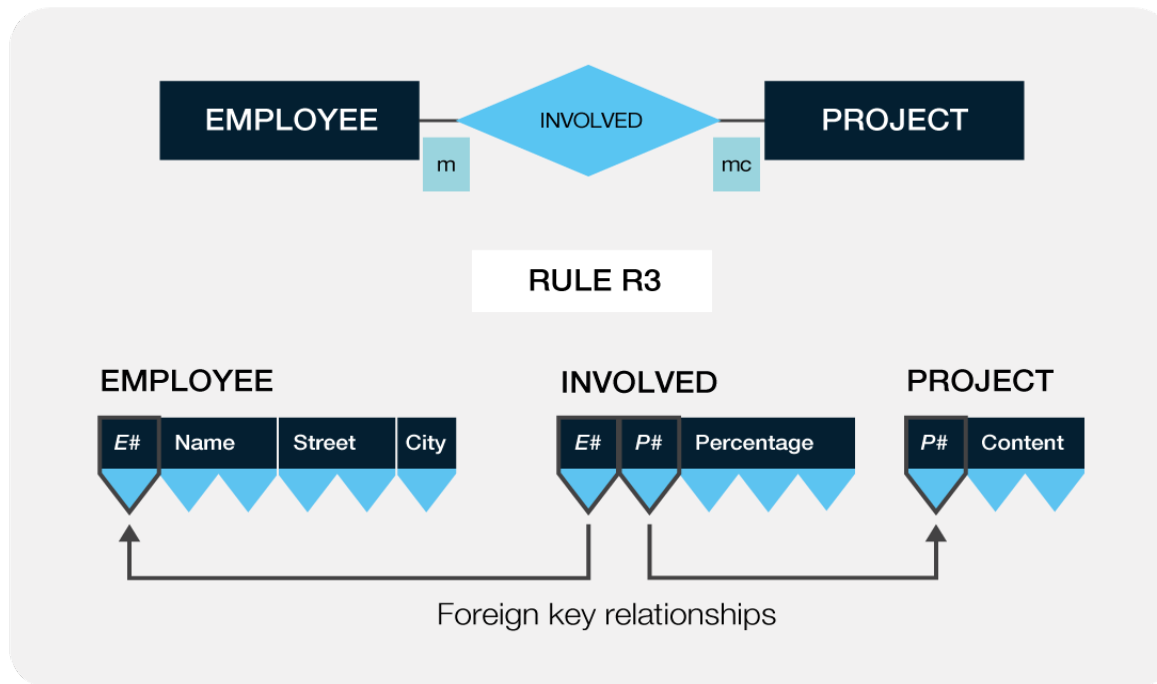


Regel R1: Jede Entitätsmenge muss als eigenständige Tabelle mit einem eindeutigen Primärschlüssel definiert werden.

Regel R2: Jede Beziehung kann als separate Tabelle definiert werden.

```
-- SQL-Code:
CREATE TABLE Department (
  D# INTEGER PRIMARY KEY,
  DeptName VARCHAR(30))
```

Zuordnungsregel für komplexe Beziehungsgruppen

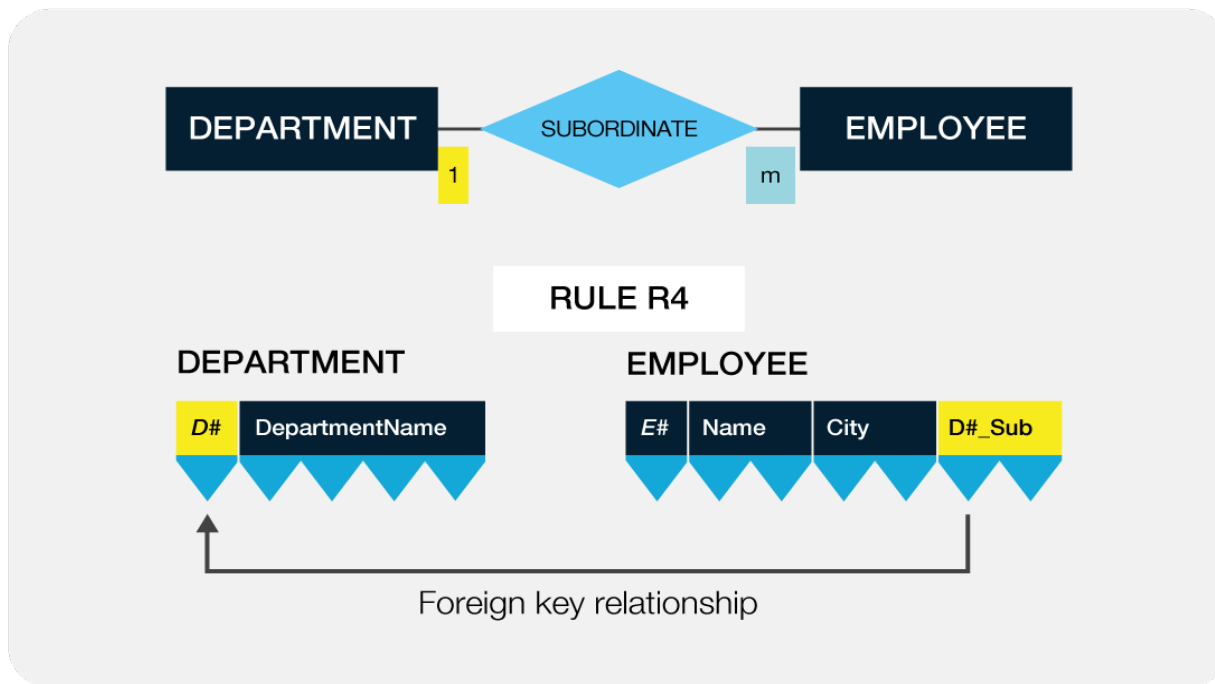


E#	P#	Perc
E19	P1	60 %
E19	P2	30
E4	P2	45

Regel R3: Jede komplex-komplexe Beziehung muss als separate Tabelle definiert werden.

```
-- SQL-Code:
CREATE TABLE involved (
  E# INTEGER REFERENCES Employee(E#),
  P# INTEGER REFERENCES Project(P#),
  Percentage DECIMAL(0.1),
  PRIMARY KEY (E#, P#) )
```

Zuordnungsregel für einfache-komplexe Beziehungsmengen



Regel R4: Einfach-komplexe Beziehungen können als eindeutige Fremdschlüssel in der referenzierenden Entitätstabelle ohne separate Beziehungstabelle abgebildet werden.

HSLU

Kaufmann & Meier 2023

EMPLOYEE (in third normal form)

E#	Name	Street	City	D#_Sub
E19	Stewart	E Main Street	Stow	D6
E1	Murphy	Morris Road	Kent	D3
E7	Howard	Lorain Ave	Cleveland	D5
E4	Bell	S Water Street	Kent	D6

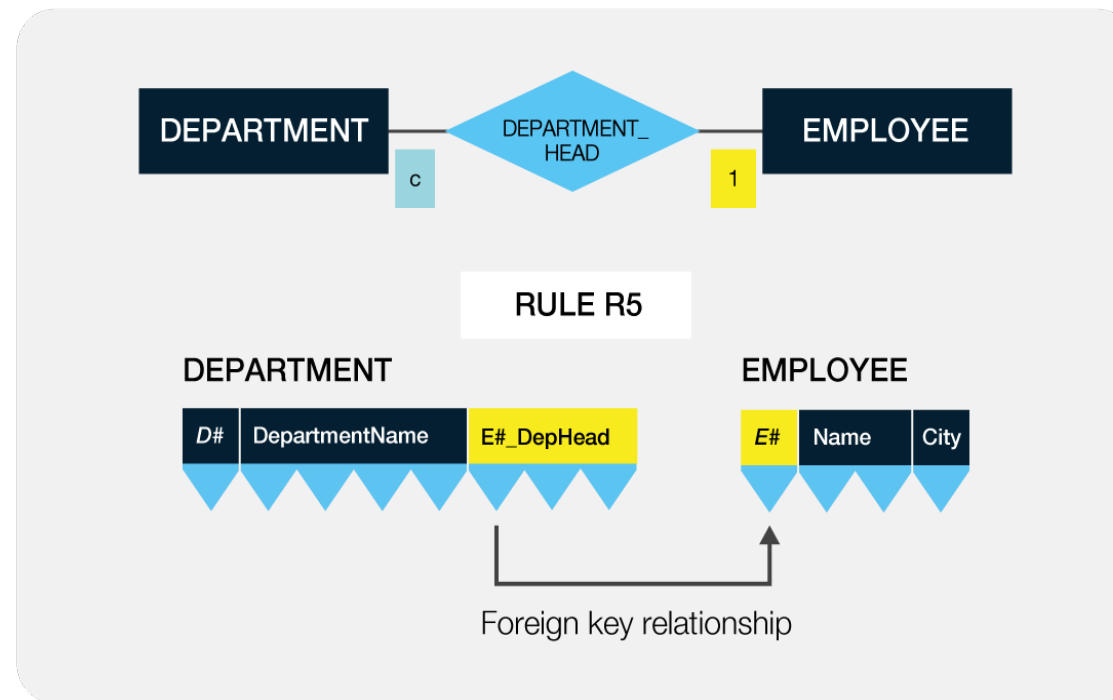
Fremdschlüssel

DEPARTMENT (3NF)

D#	DepartmentName
D3	IT
D5	HR
D6	Accounting

```
-- SQL-Code:
CREATE TABLE Employee (
  E# INTEGER PRIMARY KEY,
  Name VARCHAR(30) NOT NULL,
  City VARCHAR(30),
  D#_Sub INTEGER REFERENCES Department(D#) )
```

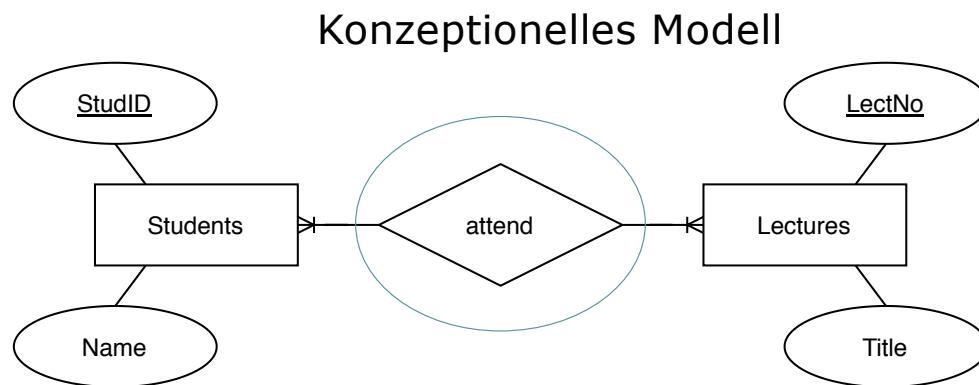
Zuordnungsregel für einfache Beziehungen



Regel R5: Einfache Beziehungen können als eindeutige Fremdschlüssel in der referenzierenden Entitätstabelle ohne separate Beziehungstabelle abgebildet werden.

```
-- SQL-Code:  
CREATE TABLE Department (  
  D# INTEGER PRIMARY KEY,  
  DeptName VARCHAR(30),  
  E#_DeptHead INTEGER  
  REFERENCES Employee(E#) )
```

Beziehung (in einem ER-Diagramm, Chen) vs. Relation (Tabelle) in RDBMS (Codd)



Wie lautet die algebraische Definition dieser Beziehungen?

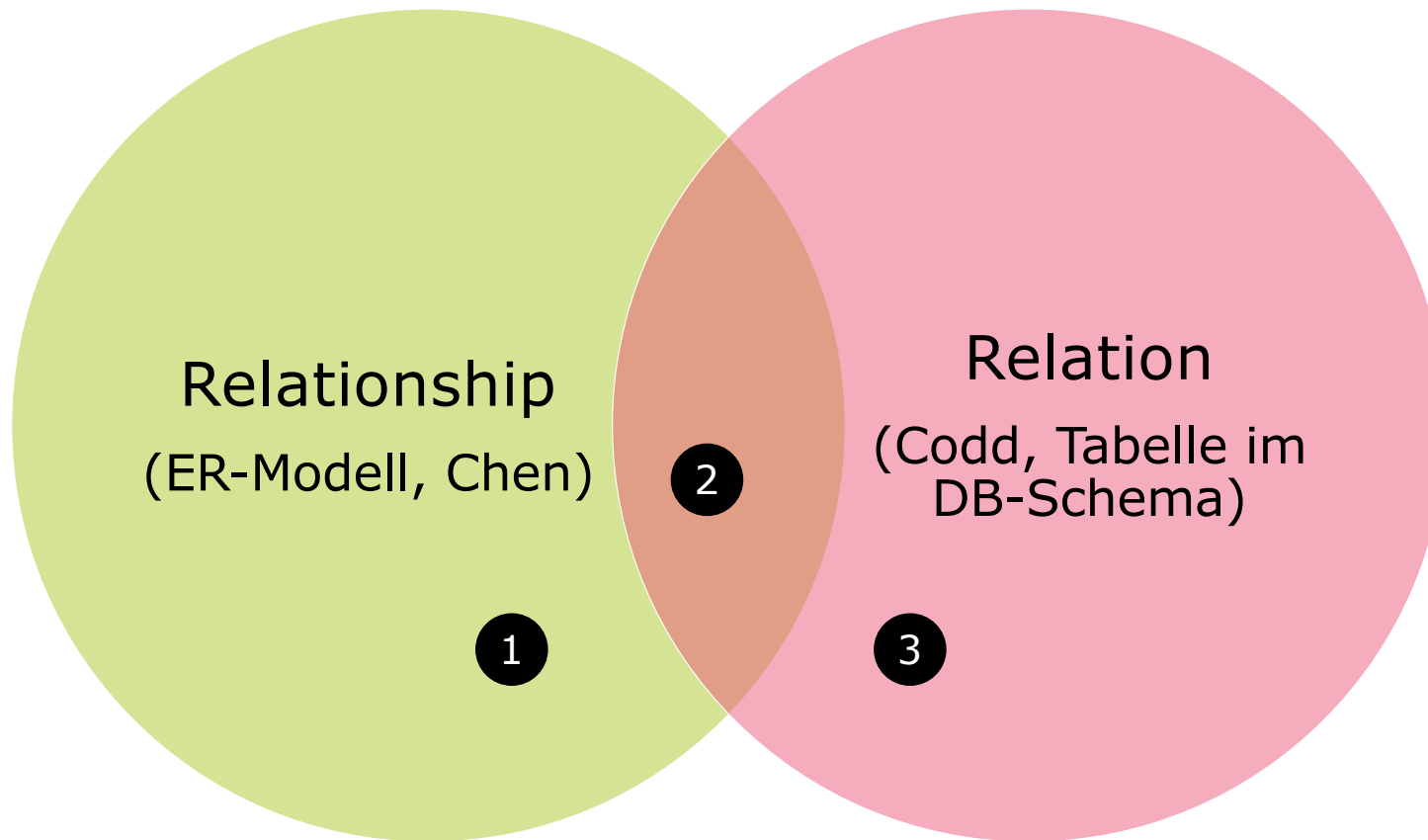
Technisches Schema (RDBMS)

Schüler	
StudID	Name
24002	Xenokrates
25403	Jonas
26120	Fichte
26830	Aristoxenos
27550	Schopenhauer
28106	Carnap
29120	Theophrastos
29555	Feuerbach

attend	
StudID	LectNo.
26120	5001
27550	5001
27550	4052
28106	5041
28106	5052
28106	5216
28106	5259
29120	5001
29120	5041
29120	5049
29555	5022
25403	5022

Eine **Beziehung** ist eine Assoziation zwischen Entitäten, die ihnen Bedeutung und Kontext verleiht.

Eine Tabelle oder **Relation** R ist eine *Teilmenge des kartesischen Produkts der Domänen*:
 $R \subseteq D_1 \times D_2 \times \dots \times D_n$ mit D_i als Domäne des i -th - Attributs. Das heißt, R ist eine Menge von n m -Tupeln:
 $R = \{t_1, t_2, \dots, t_n\}$ wobei jedes Tupel eine Liste von m Datenwerten ist $t_j = (v_1, v_2, \dots, v_m)$



Interaktion

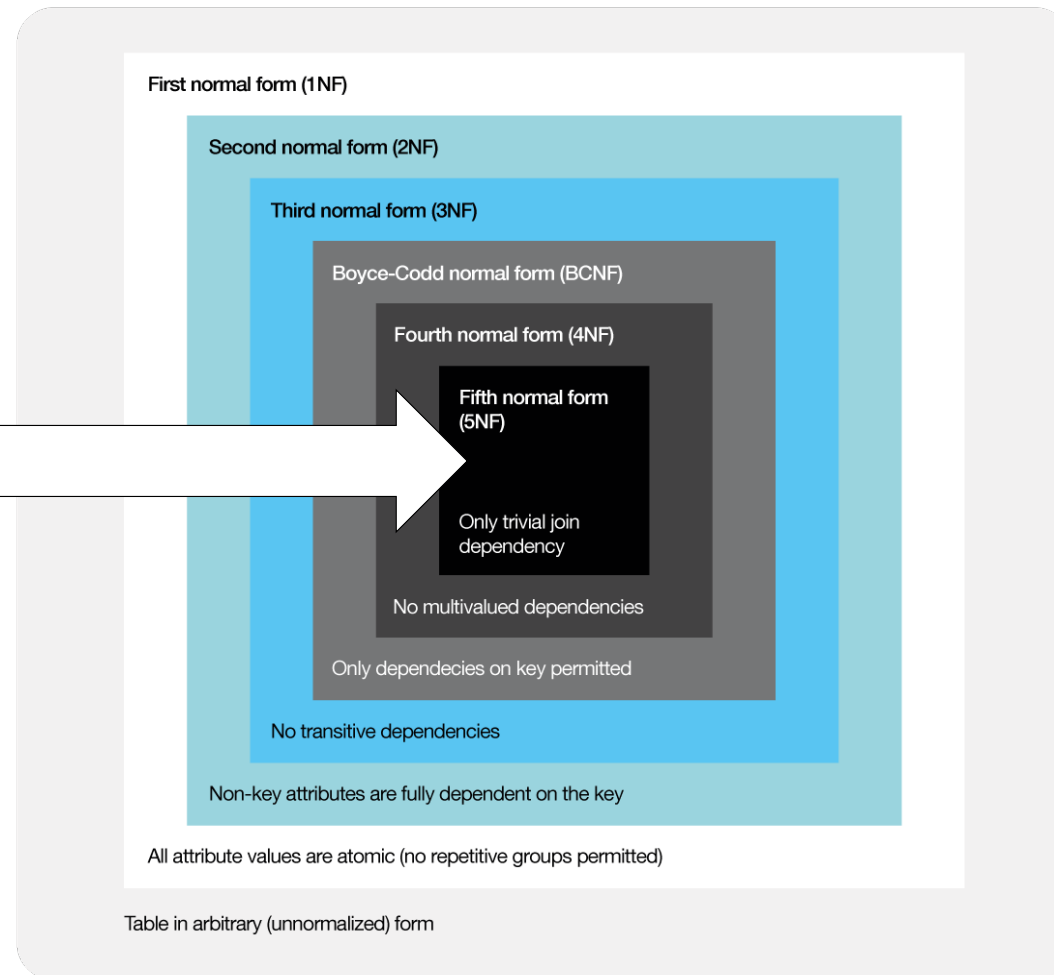
Wenn konzeptionelle ER-Modelle auf Tabellen abgebildet werden, werden einige, aber nicht alle Beziehungen als Relationen abgebildet. Fragen:

- * 1. Welche Beziehungen werden nicht als Relationen gespeichert?
- * 2. Welche Beziehungen werden als Relationen gespeichert?
- * 3. Welche Relationen speichern keine Beziehungen?

Normalformen



<https://openclipart.org/detail/17170/>



Die erste Normalform (1NF)

Atomare Daten in den Tabellenzeilen

Keine Arrays, Mengen, Aufzählungen, Listen
usw. als Attributwerte
(z. B. im Gegensatz zu OOP)

Was Codd einfach als „Normalisierung“
bezeichnete, wird heute als „erste“ Normalform
bezeichnet.

PROJECT_EMPLOYEE (unnormalized)

E#	Name	City	P#
E7	Howard	Cleveland	[P1, P9]
E1	Murphy	Kent	[P7, P11, P9]

PROJECT EMPLOYEE (first normal form)

E#	P#	Name	City
E7	P1	Howard	Cleveland
E7	P9	Howard	Cleveland
E1	P7	Murphy	Kent
E1	P11	Murphy	Kent
E1	P9	Murphy	Kent

Die zweite Normalform (2NF)

- 1.) Erste Normalform okay
- 2.) Alle Nicht-Schlüsselattribute sind vollständig funktional von den Schlüsselattributen abhängig.

Funktionale Abhängigkeit $A \rightarrow B$:
Jeder Wert von A bestimmt eindeutig den Wert von B

Wenn A ein zusammengesetzter Schlüssel ist: **Vollständige funktionale Abhängigkeit** bedeutet, dass B nur von A abhängt, nicht aber von Teilmengen von A (Unterschlüssel)

HSLU

Kaufmann & Meier 2023

EMPLOYEE (2NF)

E#	Name	City
E7	Howard	Cleveland
E1	Murphy	Kent

INVOLVED (2NF)

E#	P#
E7	P1
E7	P9
E1	P7
E1	P11
E1	P9

Redundanz und Normalformen in Tabellen

Redundanz in Tabellen kann zu *Anomalien* in Datenbanken führen

Ein Attribut ist *redundant*, wenn Werte davon **ohne Informationsverlust** weggelassen werden können.

Normalformen dienen dazu, Redundanzen zu vermeiden.

Anomalien: Einfügeanomalie, Mutationsanomalie, Löschanomalie

DEPARTMENT_EMPLOYEE

E#	Name	Street	City	D#	DepartmentName
E19	Stewart	E Main Street	Stow	D6	Accounting
E1	Murphy	Morris Road	Kent	D3	IT
E7	Howard	Lorain Avenue	Cleveland	D5	HR
E4	Bell	S Water Street	Kent	D6	Accounting

Was ist hier redundant?

In welcher Normalform befindet sich diese Tabelle und warum?

Die dritte Normalform (3NF)

- 1.) 1NF okay.
- 2.) 2NF okay.
- 3.) Die Tabelle hat **keine** transitiven Abhängigkeiten.

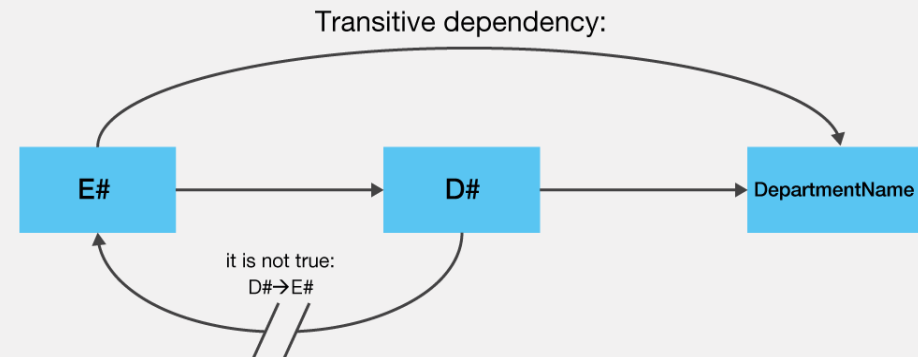
Transitive Abhängigkeit

$A \rightarrow B \rightarrow C$:

$A \rightarrow B$, $B \rightarrow C$; aber nicht $B \rightarrow A$

DEPARTMENT EMPLOYEE (in second normal form)

E#	Name	Street	City	D#	DepartmentName
E19	Stewart	E Main Street	Stow	D6	Accounting
E1	Murphy	Morris Road	Kent	D3	IT
E7	Howard	Lorain Avenue	Cleveland	D5	HR
E4	Bell	S Water Street	Kent	D6	Accounting



EMPLOYEE (in third normal form)

E#	Name	Street	City	D#_Sub
E19	Stewart	E Main Street	Stow	D6
E1	Murphy	Morris Road	Kent	D3
E7	Howard	Lorain Ave	Cleveland	D5
E4	Bell	S Water Street	Kent	D6

DEPARTMENT (3NF)

D#	DepartmentName
D3	IT
D5	HR
D6	Accounting

Die Eckpfeiler des Datenmanagements: Beispiel für ein Filmempfehlungssystem

Datennutzung

- ▶ Anwendungsfall: Filmempfehlung
- ▶ Entscheidungshilfe: Welchen Film soll ich mir ansehen?
- ▶ Datenquellen: Filme, Bewertungen (MovieLens)
- ▶ Datenvisualisierung: BI-Tool (Metabase)
- ▶ Benutzerinteraktion: Eingabe des Films, Ausgabe der Empfehlungen

Datentechnologie

- ▶ Datenbanksoftware: MySQL Server, MySQL Workbench
- ▶ Integriertes System: Metabase mit SQL-API
- ▶ Dateneingabe: MySQL LOAD
- ▶ Datenverarbeitung: SQL SELECT-Abfragen, Transformation mit INSERT INTO ... SELECT
- ▶ Leistung: Materialisierung: CREATE TABLE AS SELECT; Indizierung: CREATE INDEX

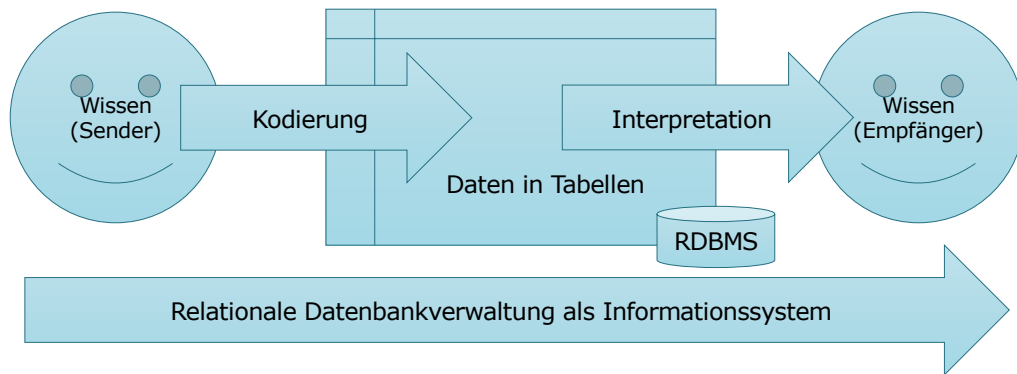
Datenverwaltung

- ▶ Operative Organisation: Entwickler, Endnutzer
- ▶ Datenzugriff:
 - ▶ Entwickler über MySQL,
 - ▶ Endnutzer über Metabase
- ▶ Zugriffsrechte:
 - ▶ Admin für Entwickler über Port 3306
 - ▶ Eingeschränkte Leserechte für Metabase

Datenarchitektur

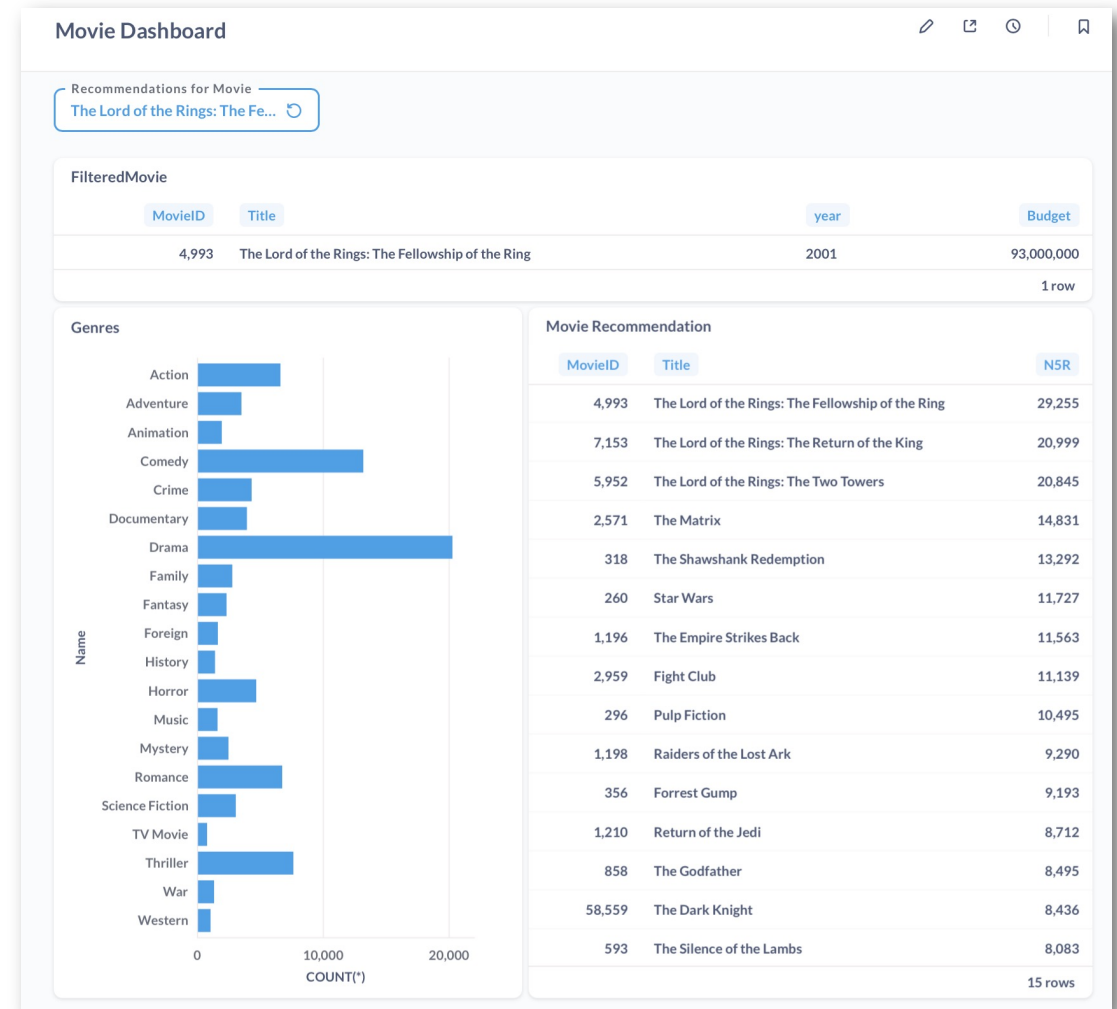
- ▶ Systemkomponenten (siehe nächste Folie)
- ▶ Datenmodellierung: (siehe nächste Folien)
 - ▶ Filme mit Titel, Genre, Jahr;
 - ▶ Bewertungen mit Benutzer, Film, Zeitstempel, Bewertung von 1 bis 5 Sternen.
- ▶ Datenbankschema: SQL CREATE TABLE

Beispiel: Filmempfehlungssystem



Filmempfehlungen
basierend auf Nutzerbewertungen

- * Entität: Film
- * Entität: Bewertung



Quelldaten

1. Wie lautet die Normalform der Bewertungsdaten?

```
PowerShell 7.3.6
PS C:\Users\micha> get-content C:\Users\micha\Downloads\ml-25m\ml-25m\ratings.csv | select -First 15
userId,movieId,rating,timestamp
1,296,5.0,1147880044
1,306,3.5,1147868817
1,307,5.0,1147868828
1,665,5.0,1147878820
1,899,3.5,1147868510
1,1088,4.0,1147868495
1,1175,3.5,1147868826
1,1217,3.5,1147878326
1,1237,5.0,1147868839
1,1250,4.0,1147868414
1,1260,3.5,1147877857
1,1653,4.0,1147868097
1,2011,2.5,1147868079
1,2012,2.5,1147868068
PS C:\Users\micha>
```

2. Wie lautet die Normalform der Filmdaten?

movies.csv - Editor					
Datei	Bearbeiten	Ansicht			
"MovieID"	"Title"	"Release_date"	"Budget"	"Genres"	"Spoken_languages"
"1"	"Toy Story"	"1995-10-30"	"30000000"	"[{\"id\": 16, \"name\": \"Animation\"}, {\"id\": 35,	
"2"	"Jumanji"	"1995-12-15"	"65000000"	"[{\"id\": 12, \"name\": \"Adventure\"}, {\"id\": 14,	
"3"	"Grumpier Old Men"	"1995-12-22"	"0"	"[{\"id\": 10749, \"name\": \"Romance\"}, {\"id	
"4"	"Waiting to Exhale"	"1995-12-22"	"16000000"	"[{\"id\": 35, \"name\": \"Comedy\"}, {\"	
"5"	"Father of the Bride Part II"	"1995-02-10"	"0"	"[{\"id\": 35, \"name\": \"Comedy\"}]"	
"6"	"Heat"	"1995-12-15"	"60000000"	"[{\"id\": 28, \"name\": \"Action\"}, {\"id\": 80, \"	
"7"	"Sabrina"	"1995-12-15"	"58000000"	"[{\"id\": 35, \"name\": \"Comedy\"}, {\"id\": 10749,	
"8"	"Tom and Huck"	"1995-12-22"	"0"	"[{\"id\": 28, \"name\": \"Action\"}, {\"id\": 12, \"	
"9"	"Sudden Death"	"1995-12-22"	"35000000"	"[{\"id\": 28, \"name\": \"Action\"}, {\"id\":	
"10"	"GoldenEye"	"1995-11-16"	"58000000"	"[{\"id\": 12, \"name\": \"Adventure\"}, {\"id\": 28,	
"11"	"The American President"	"1995-11-17"	"62000000"	"[{\"id\": 35, \"name\": \"Comedy\"}	
"12"	"Dracula: Dead and Loving It"	"1995-12-22"	"0"	"[{\"id\": 35, \"name\": \"Comedy\"}, {\"	
"13"	"Balto"	"1995-12-22"	"0"	"[{\"id\": 10751, \"name\": \"Family\"}, {\"id\": 16, \"nam	
"14"	"Nixon"	"1995-12-22"	"44000000"	"[{\"id\": 36, \"name\": \"History\"}, {\"id\": 18, \"	
"15"	"Cutthroat Island"	"1995-12-22"	"98000000"	"[{\"id\": 28, \"name\": \"Action\"}, {\"	
"16"	"Casino"	"1995-11-22"	"52000000"	"[{\"id\": 18, \"name\": \"Drama\"}, {\"id\": 80, \"n	
"17"	"Sense and Sensibility"	"1995-12-13"	"16500000"	"[{\"id\": 18, \"name\": \"Drama\"}, {\"i	

Konzeptionelles Datenmodell

Entity: Movie

Attributes:

- Movie ID
- Title
- Release Date
- Budget
- Genres
- Spoken Languages

Relationship: Movie ID

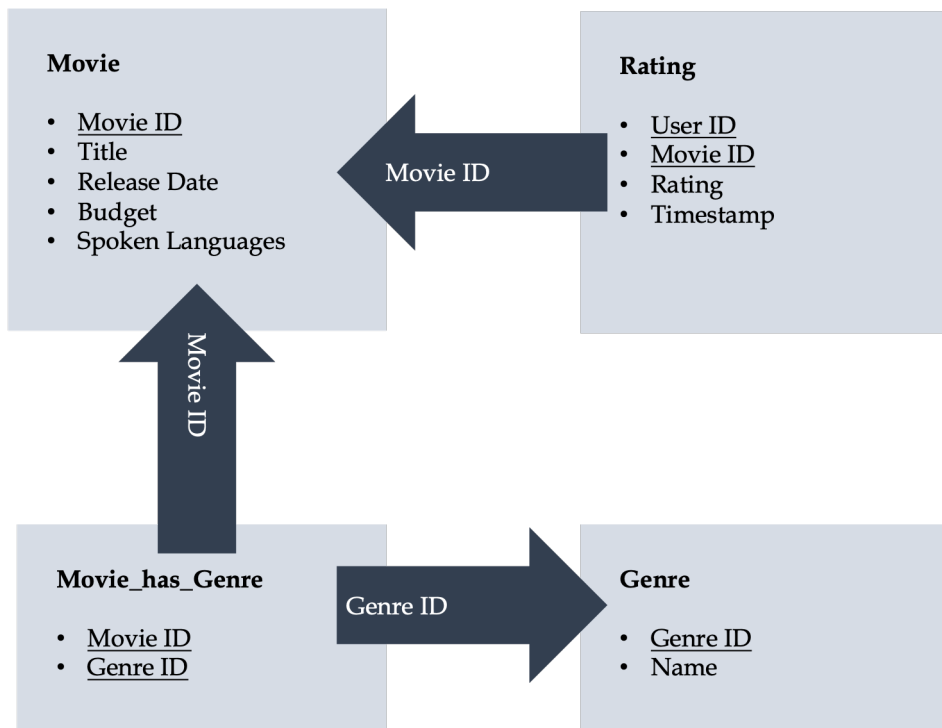
Entity: Rating

Attributes:

- User ID
- Movie ID
- Rating
- Timestamp

Normalisiertes konzeptionelles Datenmodell und Datenbankschema (3NF) für ein Filmempfehlungssystem.

Konzeptionelles Datenmodell



Datenbankschema (SQL)

```
CREATE TABLE Movies (
  MovieID INTEGER PRIMARY KEY,
  Title TEXT,
  Release_date DATE,
  Budget INTEGER,
  Sprachen JSON );
```

```
CREATE TABLE Bewertungen (
  Benutzer-ID INTEGER,
  Film-ID INTEGER,
  Bewertung INTEGER,
  Timestamp DATETIME,
  PRIMARY KEY (FilmID,
  BenutzerID),
  FOREIGN KEY (FilmID)
  REFERENCES Movies (MovieID));
```

```
CREATE TABLE Movie_has_Genre (
  MovieID INTEGER,
  GenreID INTEGER,
  PRIMARY KEY (MovieID, GenreID),
  FOREIGN KEY (MovieID)
  REFERENCES Movies (MovieID),
  FOREIGN KEY (GenreID)
  REFERENCES Genres (GenreID));
```

```
CREATE TABLE Genres (
  GenreID INTEGER PRIMARY KEY,
  Name TEXT);
```

Vielen Dank!
Fragen?

Hochschule Luzern
Informatik
Forschung
Prof. Dr. Michael Kaufmann
Dozent

T direkt +41 41 757 68 48
m.kaufmann@hslu.ch